Kigali Independent University (ULK)
Data Science

REPORT 2

# UML Class Diagram Report

## Nyalatna AI-Powered Training Center Management System

Nyalatna Capacity Building Center, Nyala, Sudan



| Group member | Student ID |
|---|---|
| Mubarak Dawood Abdalla Bakheet | 202510537 |
| Mustafa Omer Mustafa Almubark | 202510734 |



**Assignment:** UML Class Diagram Modelling Assignment

**Lecturer:** Eric Maniraguha

**Date:** 19 October 2026

**Repository:** https://github.com/mbarkdlwd-blip/uml-class-diagram-assignment

# 1. Introduction

This report presents the UML class diagram of the Nyalatna AI-Powered Training Center Management System, the information system proposed for Nyalatna Capacity Building Center. It lists every class with its attributes and methods, describes each relationship and its multiplicity, and explains the main design decisions. The editable diagram (draw.io) and the exported PNG are in the `diagrams/` folder of the repository.

# 2. Notation Used

| Symbol | Meaning |
|---|---|
| **-** / **+** | Private attribute / public method (visibility). |
| *Italic name*, «abstract» | Abstract class: cannot be instantiated directly. |
| Line with hollow triangle | Generalization (inheritance): child → parent. |
| Line with filled diamond | Composition: the part cannot exist without the whole (diamond at the whole). |
| Plain line | Association between two independent classes. |
| Dashed arrow «use» | Dependency: the class uses an enumeration type. |
| Line with open arrowhead | Directed association: navigable only in the arrow's direction. |
| Folded-corner note { … } | Constraint: a business rule that applies to the attached class. |
| 1, 0..1, 0..*, 1..* | Multiplicity: exactly one, zero or one, zero or many, one or many. |

# 3. Complete UML Class Diagram

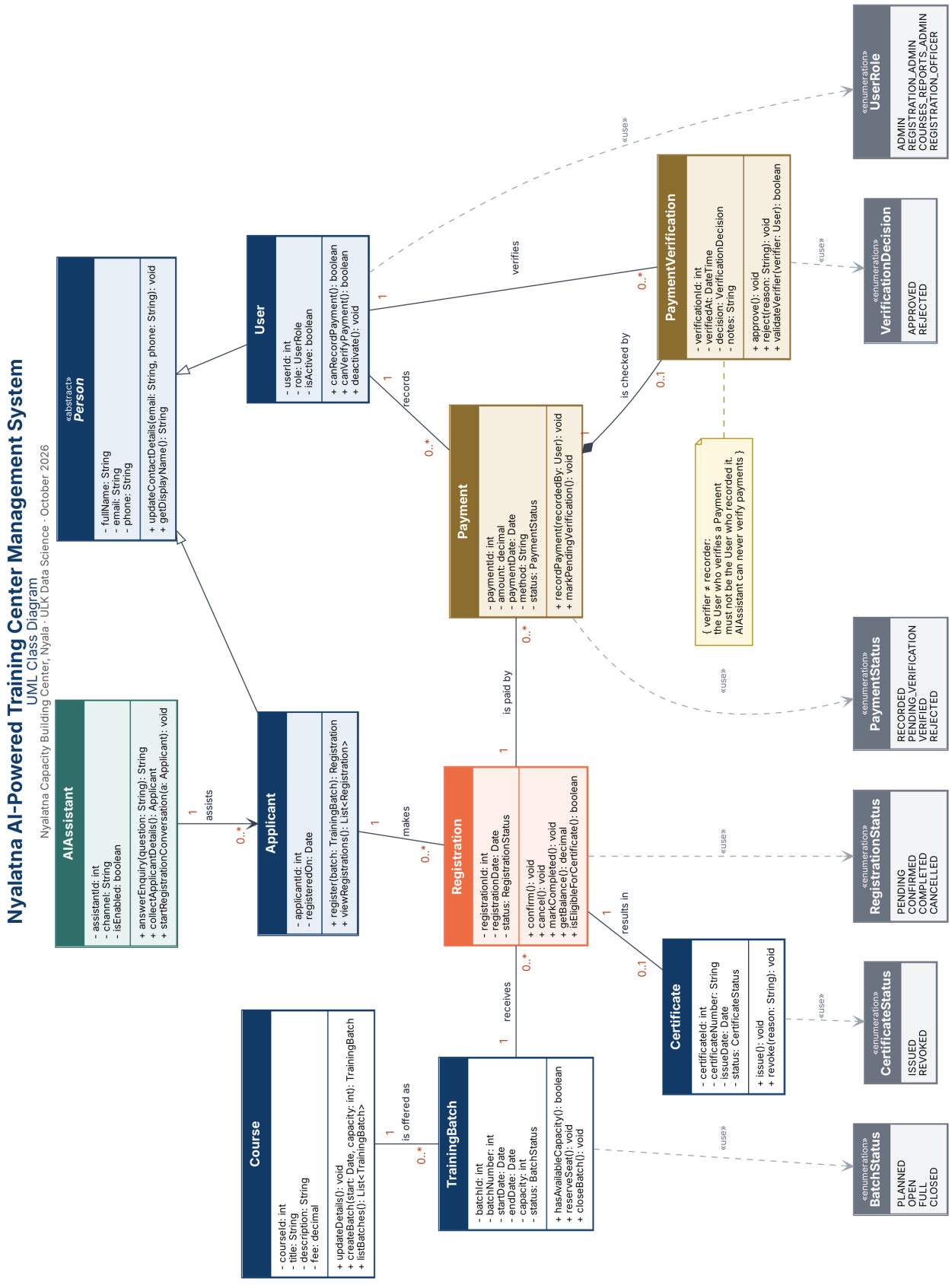



Figure 1 – UML class diagram of the Nyalatna AI-Powered Training Center Management System (rotated; full-size PNG in diagrams/)

# 4. Classes, Attributes and Methods

## 4.1 Person (abstract)

Abstract parent holding the name and contact details shared by applicants and staff users.

| Attributes | Methods (operations) |
|---|---|
| - fullName: String<br>- email: String<br>- phone: String | + updateContactDetails(email: String, phone: String): void<br>+ getDisplayName(): String |

## 4.2 Applicant

A person applying for or attending a course. Registers in batches and follows their status. Inherits all attributes and methods of Person.

| Attributes | Methods (operations) |
|---|---|
| - applicantId: int<br>- registeredOn: Date | + register(batch: TrainingBatch): Registration<br>+ viewRegistrations(): List<Registration> |

## 4.3 User

A staff account with a role. Its role decides whether it may record or verify payments. Inherits all attributes and methods of Person.

| Attributes | Methods (operations) |
|---|---|
| - userId: int<br>- role: UserRole<br>- isActive: boolean | + canRecordPayment(): boolean<br>+ canVerifyPayment(): boolean<br>+ deactivate(): void |

## 4.4 Course

A training programme in the catalogue, with its description and fee.

| Attributes | Methods (operations) |
|---|---|
| - courseId: int<br>- title: String<br>- description: String<br>- fee: decimal | + updateDetails(): void<br>+ createBatch(start: Date, capacity: int): TrainingBatch<br>+ listBatches(): List<TrainingBatch> |

## 4.5 TrainingBatch

One scheduled offering (cohort) of a course, with dates, capacity and status.

| Attributes | Methods (operations) |
|---|---|
| - batchId: int<br>- batchNumber: int<br>- startDate: Date<br>- endDate: Date<br>- capacity: int<br>- status: BatchStatus | + hasAvailableCapacity(): boolean<br>+ reserveSeat(): void<br>+ closeBatch(): void |

### 4.6 Registration

Links one applicant to one training batch and tracks the registration status and balance.

| Attributes | Methods (operations) |
|---|---|
| - registrationId: int<br>- registrationDate: Date<br>- status: RegistrationStatus | + confirm(): void<br>+ cancel(): void<br>+ markCompleted(): void<br>+ getBalance(): decimal<br>+ isEligibleForCertificate(): boolean |

### 4.7 Payment

A payment (full or instalment) recorded by a staff user against a registration.

| Attributes | Methods (operations) |
|---|---|
| - paymentId: int<br>- amount: decimal<br>- paymentDate: Date<br>- method: String<br>- status: PaymentStatus | + recordPayment(recordedBy: User): void<br>+ markPendingVerification(): void |

### 4.8 PaymentVerification

The independent review of a payment by a second staff user, with the decision and notes.

| Attributes | Methods (operations) |
|---|---|
| - verificationId: int<br>- verifiedAt: DateTime<br>- decision: VerificationDecision<br>- notes: String | + approve(): void<br>+ reject(reason: String): void<br>+ validateVerifier(verifier: User): boolean |

### 4.9 Certificate

Proof of completion issued for an eligible, completed registration.

| Attributes | Methods (operations) |
|---|---|
| - certificateId: int<br>- certificateNumber: String<br>- issueDate: Date<br>- status: CertificateStatus | + issue(): void<br>+ revoke(reason: String): void |

### 4.10 AIAssistant

Answers applicant enquiries and collects registration details. It has no access to payment verification.

| Attributes | Methods (operations) |
|---|---|
| - assistantId: int<br>- channel: String<br>- isEnabled: boolean | + answerEnquiry(question: String): String<br>+ collectApplicantDetails(): Applicant<br>+ startRegistrationConversation(a: Applicant): void |

### 4.11 Enumerations

| Enumeration | Values | Used by |
|---|---|---|
| **BatchStatus** | PLANNED, OPEN, FULL, CLOSED | TrainingBatch.status |
| **RegistrationStatus** | PENDING, CONFIRMED, COMPLETED, CANCELLED | Registration.status |
| **PaymentStatus** | RECORDED, PENDING_VERIFICATION, VERIFIED, REJECTED | Payment.status |
| **VerificationDecision** | APPROVED, REJECTED | PaymentVerification.decision |
| **UserRole** | ADMIN, REGISTRATION_ADMIN, COURSES_REPORTS_ADMIN, REGISTRATION_OFFICER | User.role |
| **CertificateStatus** | ISSUED, REVOKED | Certificate.status |

## 5. Relationships and Multiplicities

Summary of all relationships in the diagram:

| # | From | To | Type | Multiplicity | Name |
|---|---|---|---|---|---|
| 1 | Applicant | Person | Generalization | — | is a |
| 2 | User | Person | Generalization | — | is a |
| 3 | Course | TrainingBatch | Association | 1 → 0..* | is offered as |
| 4 | Applicant | Registration | Association | 1 → 0..* | makes |
| 5 | TrainingBatch | Registration | Association | 1 → 0..* | receives |
| 6 | Registration | Payment | Association | 1 → 0..* | is paid by |
| 7 | Payment | PaymentVerification | Composition | 1 → 0..1 | is checked by |
| 8 | User | Payment | Association | 1 → 0..* | records |
| 9 | User | PaymentVerification | Association | 1 → 0..* | verifies |
| 10 | Registration | Certificate | Association | 1 → 0..1 | results in |
| 11 | AIAssistant | Applicant | Directed assoc. | 1 → 0..* | assists |

## 6. Explanation of the Relationships

**Applicant, User → Person** — Generalization (inheritance); multiplicity —

Applicants and staff users are both people with a name, email and phone. They inherit these from the abstract Person class and add their own attributes: an applicant has registrations, a user has a role.

**Course — TrainingBatch** — Association; multiplicity 1 to 0..*

A course can be run as many batches over time, or none yet. Each batch belongs to exactly one course.

**Applicant — Registration — TrainingBatch** — Associations (resolve many-to-many); multiplicity 1 to 0..* on each side

An applicant can register for many batches and a batch receives many applicants. Registration sits between them and stores the data of that link: date and status. Each registration belongs to exactly one applicant and one batch.

**Registration — Payment** — Association; multiplicity 1 to 0..*

Fees may be paid in full or in instalments, so a registration has zero or many payments. Each payment belongs to one registration.

**Payment ◆— PaymentVerification** — Composition; multiplicity 1 to 0..1

A payment is verified at most once (0 while it is still pending). The verification has no meaning without its payment, so it is a composition.

**User — Payment (records)** — Association; multiplicity 1 to 0..*

Each payment is recorded by exactly one staff user; a user may record many payments.

**User — PaymentVerification (verifies)** — Association; multiplicity 1 to 0..*

Each verification is performed by exactly one staff user. The constraint note on the diagram states that this user must be different from the user who recorded the payment.

**Registration — Certificate** — Association; multiplicity 1 to 0..1

A certificate is issued only after the registration is completed, paid and eligible, so a registration has at most one certificate.

**AIAssistant → Applicant** — Directed association; multiplicity 1 to 0..*

The assistant serves many applicants and can see their details (navigable towards Applicant). It has no link to Payment or PaymentVerification, which shows in the model that it cannot touch payment decisions.

**Classes ⇢ enumerations** — Dependency / typed attribute; multiplicity —

Status, decision and role attributes use enumerations, so they can only take the listed values.

## 7. Business Rules (Constraints)

- The staff user who verifies a payment must be different from the user who recorded it (verifier ≠ recorder).
- Only users whose role allows it may record or verify payments; deactivated users can do neither.
- The AI assistant can never verify payments; it has no association with Payment or PaymentVerification.
- A registration is refused when the training batch has no available capacity.
- A certificate is issued only for a completed, fully paid registration.

## 8. Design Decisions

**Abstract Person class.** Applicant and User shared fullName, email and phone. Moving them to an abstract parent removes duplication and shows inheritance.

**Registration as a linking class.** Registration resolves the many-to-many relationship between Applicant and TrainingBatch. Payments and the certificate hang off Registration because they belong to one applicant in one specific batch.

**Separation of duties.** Payment and PaymentVerification are separate classes, each linked to its own User. This is what makes the rule 'the recorder cannot verify' expressible in the model; the rule itself is written as a UML constraint note.

**Limited AI assistant.** AIAssistant is only associated with Applicant. Giving it no association to the payment classes is a design decision that keeps financial approval with human staff.

**Enumerations.** Every status, decision and role uses an enumeration so invalid values cannot be stored.

## 9. Conclusion

The class diagram captures the full applicant journey at Nyalatna Center: an applicant, helped by the AI assistant, registers in a batch of a course, pays fees that are recorded by one staff user and verified by another, and receives a certificate. The model is built around the Registration class, uses inheritance to avoid duplication, and expresses the separation-of-duties rule as a UML constraint, which makes it a sound base for implementing the system later.